



**BANGLADESH UNIVERSITY OF ENGINEERING
AND TECHNOLOGY**

CSE 306

Computer Architecture Sessional

**Assignment-3: 4-bit MIPS Design, Simulation, and
Implementation**

Section - B1

Group - 03

Members of the Group:

1. 2005071 – Sagor Chanda
2. 2005069 – Souvik Mandol
3. 2005076 – S. M. Kausar Parvej
4. 2005068 – Md. Sumon Hossain
5. 1705083 – Hozifa Rahman Hamim

1. Introduction

MIPS (Microprocessor without Interlocked Pipeline Stages) is a RISC (Reduced Instruction Set Computer) architecture widely used in embedded systems, networking devices, and other applications requiring efficient processing power. Its fixed instruction set ensures regularity and simplicity in instruction execution. Here is an example of add instruction:

Operation	Example	Instruction	Corresponding Action
Addition	$x = a + b$	add \$t2, \$t1, \$t3	$\$t2 = \$t1 + \$t3$

In MIPS, instructions are typically executed in a pipeline fashion, where each stage of the pipeline handles a different aspect of instruction execution, such as instruction fetch, decode, execute, memory access, and write back. This pipelining allows for efficient processing of multiple instructions simultaneously.

The MIPS datapath consists of various components, including registers, ALUs (Arithmetic Logic Units), MUXs (Multiplexers), memories, and control elements, which work together to process instructions efficiently. The datapath is responsible for executing instructions by performing operations such as arithmetic and logic operations, memory accesses, and control transfers.

2. Instruction Set

In this assignment we are designing a 4-bit MIPS which is consists of:

- 8-bits Address bus
- 4-bits Data bus
- 4-bit ALU
- Temporary registers are: \$zero, \$t0, \$t1, \$t2, \$t3, \$t4

Instruction set for this MIPS is given below:

Instruction ID	Category	Type	Instruction
A	Arithmetic	R	add
B	Arithmetic	I	addi
C	Arithmetic	R	sub
D	Arithmetic	I	subi
E	Logic	R	and
F	Logic	I	andi
G	Logic	R	or
H	Logic	I	ori
I	Logic	S	sll
J	Logic	S	srl
K	Logic	R	nor
L	Memory	I	sw
M	Memory	I	lw
N	Control-conditional	I	beq
O	Control-conditional	I	bneq
P	Control-unconditional	J	j

Table 1: Instruction Set Description

Our MIPS Instructions will be 16-bits long with the following four formats

<i>R-type</i>	Opcode 4-bits	Src Reg-1 4-bits	Src Reg-2 4-bits	Dst Reg 4-bits
<i>S-type</i>	Opcode 4-bits	Src Reg-1 4-bits	Dst Reg 4-bits	Shamt 4-bits
<i>I-type</i>	Opcode 4-bits	Src Reg-1 4-bits	Src Reg-2/Dst Reg 4-bits	Address/Immediate 4-bits
<i>J-type</i>	Opcode 4-bits	Target Jump Address 8-bits		0 4-bits

Table 2: Instruction Format

Opcodes of the instructions are of 4 bits, so between 0 and 15. We're given the Instruction assignment **POJMBNDECAGIHKLF**. So, our instruction set would be something like this:

Opcode	Instruction ID	Category	Type	Instruction
0000	P	Control-unconditional	J	j
0001	O	Control-conditional	I	bneq
0010	J	Logic	S	srl
0011	M	Memory	I	lw
0100	B	Arithmetic	I	addi
0101	N	Control-conditional	I	beq
0110	D	Arithmetic	I	subi
0111	E	Logic	R	and
1000	C	Arithmetic	R	sub
1001	A	Arithmetic	R	add
1010	G	Logic	R	or
1011	I	Logic	S	sll
1100	H	Logic	I	ori
1101	K	Logic	R	nor
1110	L	Memory	I	sw
1111	F	Logic	I	andi

Table 3: Instruction for section B1-Group 3

3. Complete Block diagram of a 4-bit MIPS processor

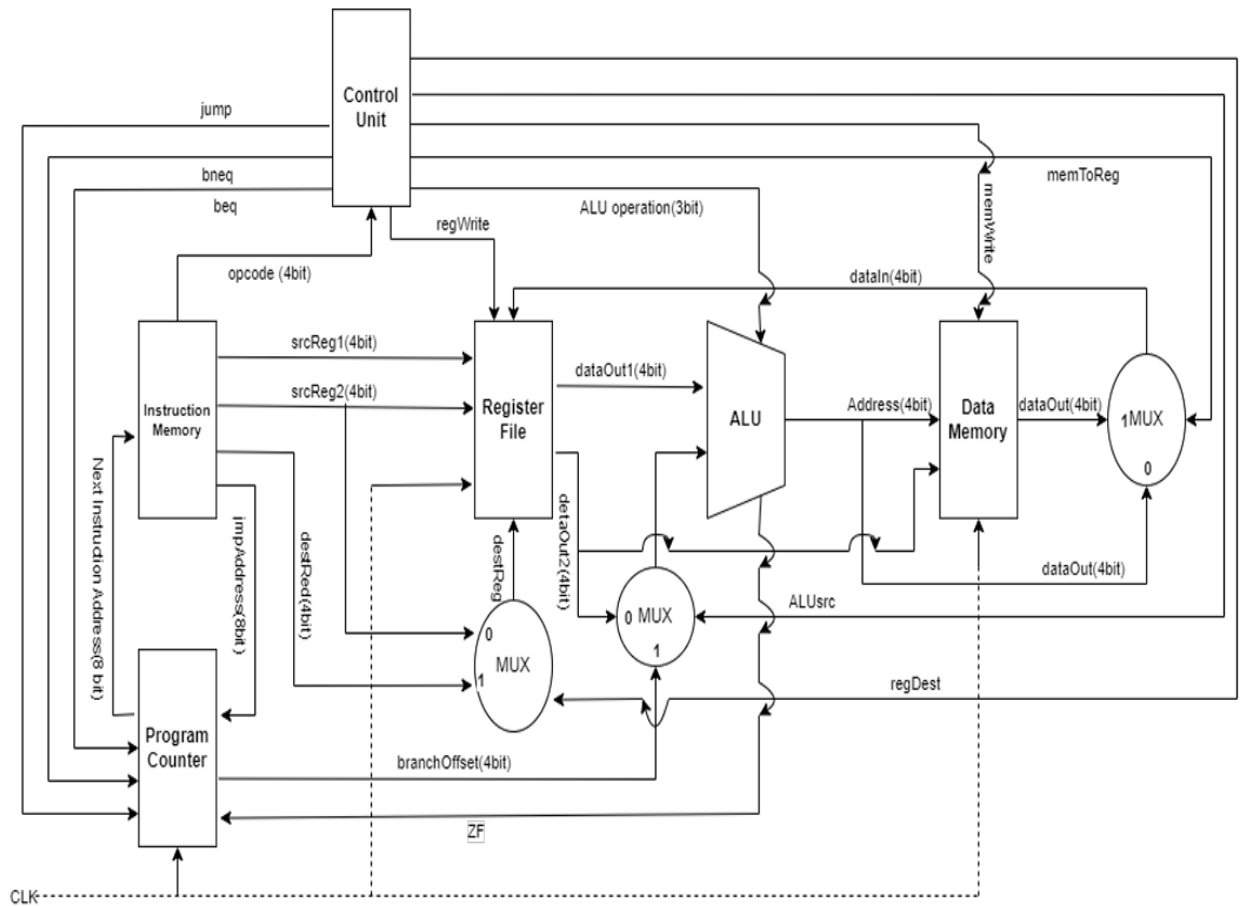


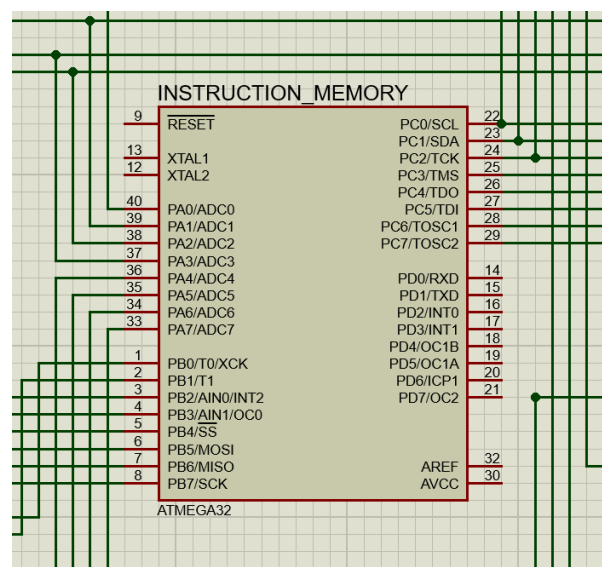
Figure: Complete Block diagram of 4-bit MIPS

4. Circuit diagrams of the main components

We will describe the design steps by listing and detailing out each of the major individual components.

4.1 Instruction Memory

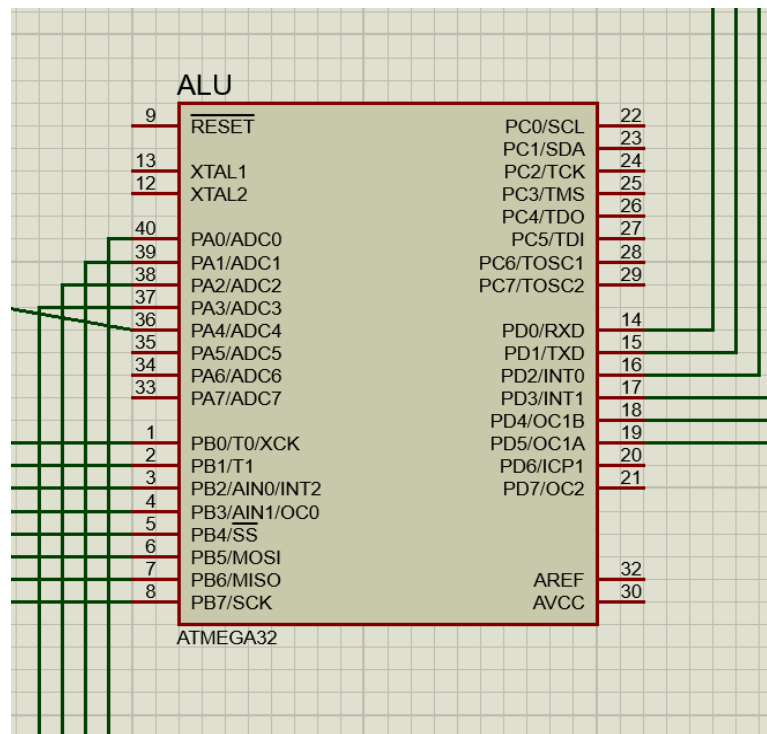
We have used ATmega32A arrays to store the instructions in our instruction memory. The instructions were provided to us as input and then converted to MIPS instruction code, resulting in 16-bit instructions. These instructions were divided into four 4-bit values: the first 4 bits represented the destination register or immediate value, the next 4 bits represented the second source register or destination register, the subsequent 4 bits represented the first source register, and the final 4 bits represented the Opcode that indicated the type of instruction. As the PC value incremented, the instruction memory outputs a new instruction.



4.2 ALU

Our next task is to design a 4-bit ALU to handle arithmetic and logical instructions. The ALU takes two input values and an ALUOP value, and produces a 4-bit output value, which is then used for further processing. Additionally, the ALU generates a zero flag that verifies whether the output is zero or not.

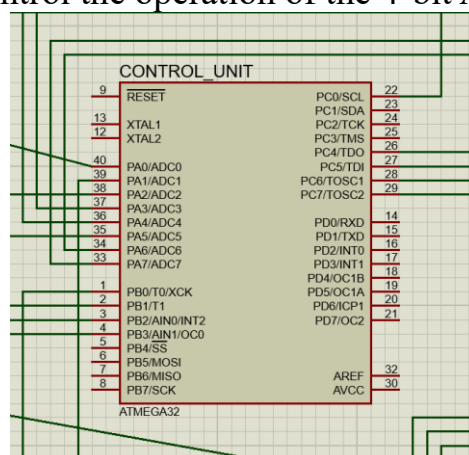
To use the ALU, our circuit takes the opcode along with two 4-bit input values. It also takes beq and bneq flags as inputs. The 4-bit ALU generates the final output value, while the zero flag, along with the beq and bneq flags (output from the control unit), produces a bsel flag that indicates whether we need to branch or not.



4.3 Control Unit:

In the control unit, we set the values for our MUX operations based on the provided instruction set for each group. To do this, we use a 16-bit ROM, where we store the hexadecimal values for the different operations.

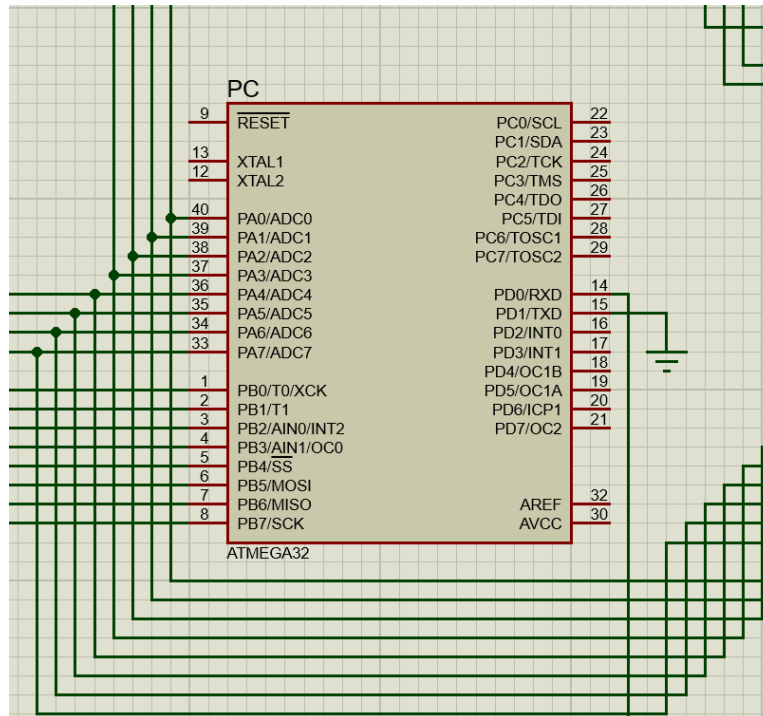
When we receive the 4-bit OpCode, we use it to select the corresponding operation from the ROM. The ROM outputs 9 selector bits for the operation, as well as a 3-bit ALUop that is used to control the operation of the 4-bit ALU.



4.4 PC Register

To store the value of our PC at any given time, we use a PC register. This register consists of 4 D-flip flops, which are used to store the PC value.

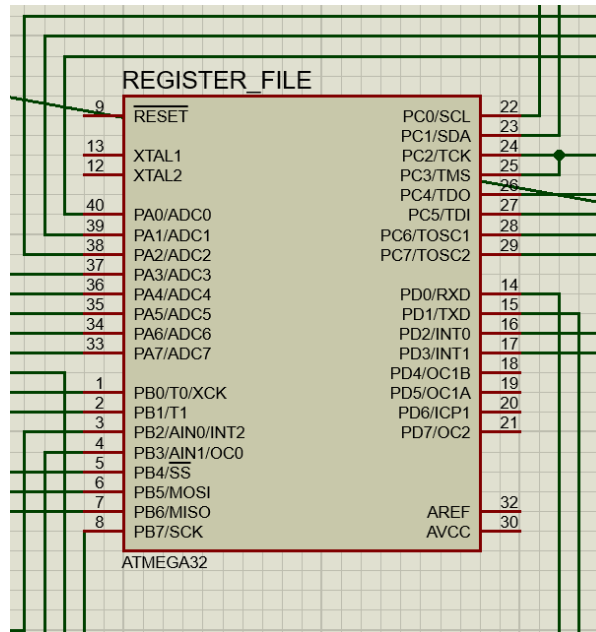
In every clock cycle, the PC register outputs the stored value of the PC, while the current PC instruction is being executed. At the same time, the PC instruction sends the next value for the PC as an input for the PC register. The next value can either be PC+1 or PC+jump amount, depending on the specific instruction being executed.



4.5 Register File

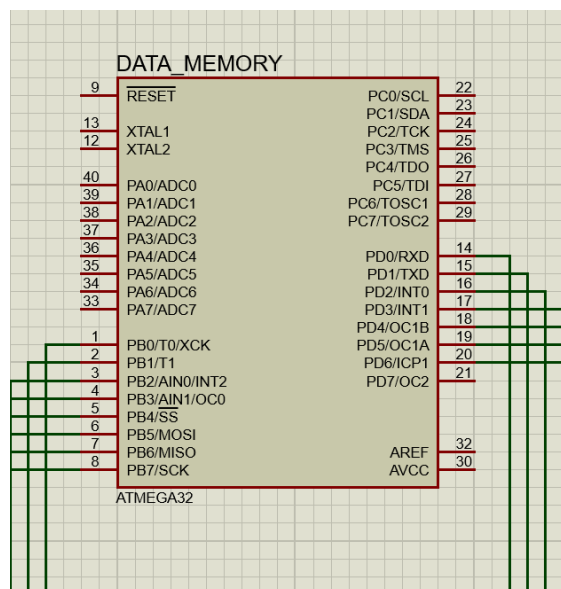
To implement our register file, we first created a file consisting of 7 registers. Among these registers, we implemented 5 temporary registers, namely \$t0, \$t1, \$t2, \$t3, and \$t4. Additionally, we used the \$sp register to handle stack push and pop instructions, and implemented the \$zero register to handle arithmetic operations with zero value.

To read from or write to a register, the register file takes the register address as an input. We also need to provide a write data value and the corresponding write register address as input when writing to a register. However, if we are executing a store word instruction, we do not need to write in the register file. Therefore, we take a selection bit, RegWrite, as an input to determine whether we need to write the value or not.

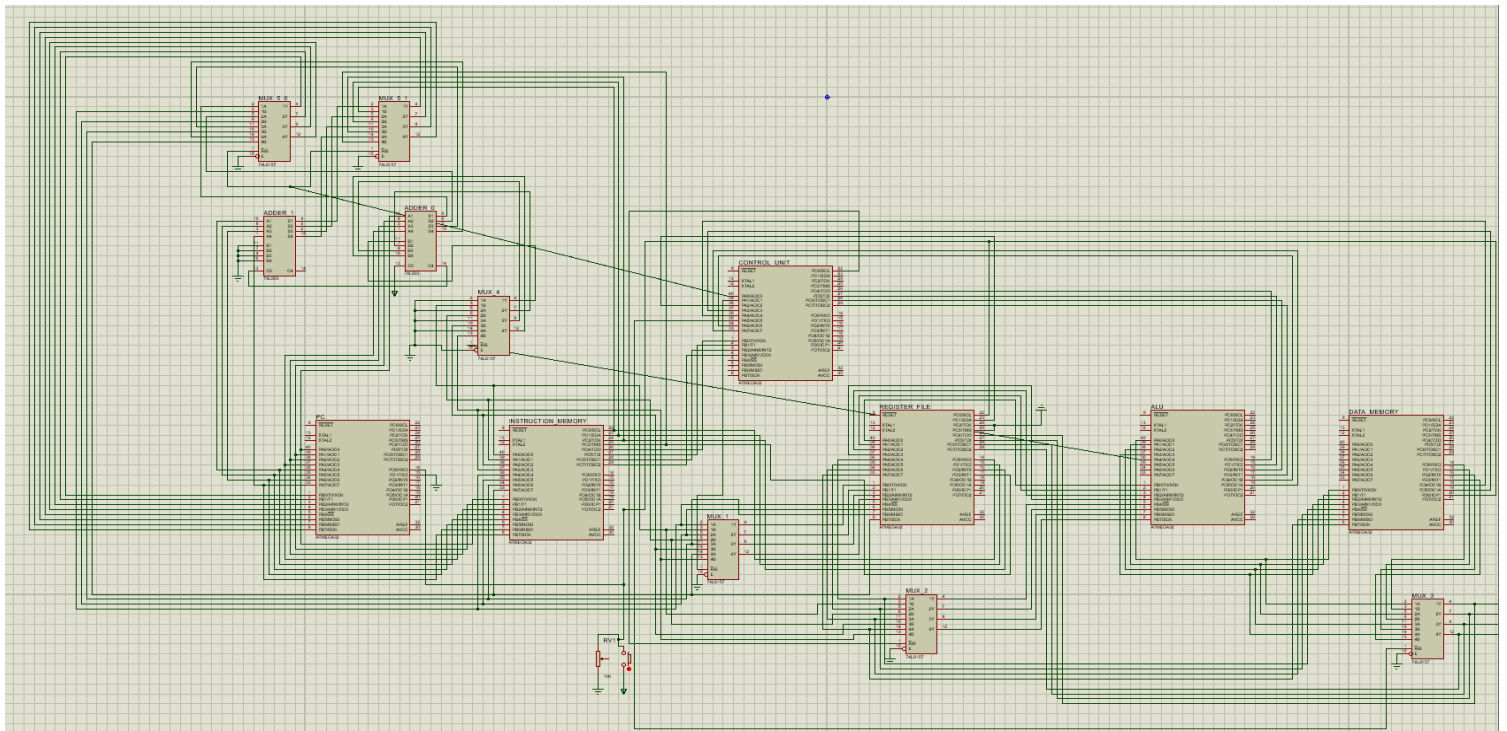


4.6 Data Memory

Data Memory is our storage for memory. It takes the memory address to write and the data to write in the 16B RAM. Now there can be read from memory in lw instruction and write in memory in store word instruction. So based on two selector bits, we select our operation either to read or write . For memory read (lw) , the output is our 4 bit memory data.



Here is our complete circuit diagram:



5. Approach to implement the push and pop instructions

We have a special register called \$sp that holds a pointer to the data memory's last location (1111). We offer a pop operation in addition to two distinct types of push operations. We use multiple fundamental operations to carry out these activities.

- **Push (push \$reg):** push \$reg and Mem[\$sp] = \$reg are interchangeable. Using the 'sw' instruction, we first put the value of \$reg in the memory pointed to by \$sp in order to do this push operation. The result is then stored in \$sp using the subi command after \$sp's value has been decreased by 1. To do this push action, two clock pulses are required.

push \$reg => sw \$reg 0(\$sp)

subi \$sp \$sp 1

- **Pop (pop \$reg):** The procedure $\$reg = \text{mem}[\$sp]$ is identical to pop \$reg. In order to do this push action, we first use the 'addi' instruction to put the result in \$sp after increasing its value by 1. Next, we use the 'lw' instruction to load the value of the memory referred to by \$sp into \$reg. Hence, two clock pulses are required for this pop operation.

**pop \$reg => addi \$sp \$sp 1
lw \$reg 0(\$reg)**

6. ICs used with their count

IC Number	IC Name	Count
ATmega 32	ATmega	6
IC 74157	Mux	6
IC 7483	Adder	2

7. The Simulator Used along with the Version Number:

----Proteus – 8.9

8. Discussion

- For implementing the circuit in software level , we used 7400-library integrated circuits. In hardware level, necessary coding has been done at ATMEGA level.
- In hardware level, we showed the value of each ALU operation and loaded memory value.
- The value of PC was also shown for each clock cycle. The value of each data memory can be shown using specific instruction.
- All of our ATmega were tested before integrating it to our final circuit.
- Wires with different sizes were used to make the whole circuit organized and easily understandable for the user.
- The outputs were checked multiple times for a large number of input test cases to ensure that our circuit satisfies the expected output values from the given input file.
- Clock pulse was provided using a Push Button Switch.

9. Contribution

ID	Contribution
2005076	Implemented Register File and Push Pop Instruction. Assembled all the discrete parts into the complete project.
2005071	Implemented Program Counter and 4-bit ALU. Also contributed in report writing.
2005069	Implemented Control Unit and assembled necessary equipment for the project.
2005068	Implemented Instruction Memory and helped in implementing 4-bit ALU.
1705083	Implemented Data Memory, ALU and contributed in report writing.